GitHub Repository Intelligence Extension

Project plan: evidence-backed repository exploration and pull-request change-impact analysis

Platform	Chrome extension (Manifest V3) on GitHub.com, plus a TypeScript analysis backend
Language scope	JavaScript and TypeScript only, file-level dependency graph first
Main outputs	Interactive dependency graph; pull-request impact report with confirmed / inferred / unverified findings
MVP length	About two weeks, followed by an evaluation phase
Status	Plan, draft 1 (October 2026)

Contents

1. Summary . . . 2

2. Context: the problem we are solving . . . 2

3. Scope and quality rules . . . 3

4. Architecture . . . 4

5. Tech stack and why each piece is there . . . 4

6. How the process unfolds: the 12-step pipeline . . . 5

7. Project folder layout . . . 10

8. Build order . . . 11

9. Evaluation: what makes it resume-worthy . . . 12

10. Real-world concepts demonstrated . . . 12

11. Risks and how to handle them . . . 13

12. Success criteria . . . 13

13. Resume line and README notes . . . 13

1. Summary

This project is a Chrome extension that adds buttons to GitHub pages. On a repository page, it draws an interactive map of how files depend on each other. On a pull request page, it lists the files and tests that may be affected by the change, and shows the proof: which file imports which, and on which line. Every finding is labeled **confirmed**, **inferred** or **unverified**. An AI model writes a short explanation, but only from facts the analyzer has already found.

> **Core design rule.** The analyzer finds facts. The LLM only explains them. The LLM never decides what depends on what.

What this is not. It is not another chatbot for asking questions about a repository. Chat about a repository already exists, for example in GitHub Copilot. The value here is a visual map, a pull-request impact trace, and an evidence trail on every claim.

2. Context: the problem we are solving

2.1 Core problem statement

> Developers struggle to understand how files and modules in an unfamiliar GitHub repository are connected, and how a change in one part of the code may affect other parts of the project.

2.2 The five developer pain points

These are commonly reported difficulties in software work. They are a **problem hypothesis to validate with developers**, not a measured ranking of what developers experience most often.

#	Pain point	What it means	Impact
1	Understanding an unfamiliar codebase	A newcomer does not know where the app starts, what each folder does, or how files work together.	Time spent reading files to build a mental map.
2	Finding dependencies between files	Changing a module without knowing who imports or depends on it.	Hidden connections, extra manual navigation.
3	Predicting the impact of a pull request	A PR shows what changed, not what it might affect.	Reviewers may overlook affected modules or tests.
4	Finding the right tests	Hard to tell which tests cover the changed code.	Slower reviews; regressions may be missed.
5	Trusting AI explanations	An explanation may miss a dependency or invent a relationship.	Developers must re-check answers against the code.

2.3 MVP priorities

- **Priority 1 - Repository understanding.** A visual file-dependency graph.
- **Priority 2 - Pull-request change-impact analysis.** Which files or modules might be affected, and which tests are relevant.
- **Priority 3 - Evidence-backed explanations.** Findings linked to source files and lines; confirmed relationships clearly separated from inferred or unverified ones.

2.4 Problem to product behavior

Developer problem	What the extension does
Too many files to understand	Builds a visual repository map.
Dependencies are hard to trace	Analyzes imports and other code relationships.
Pull-request impact is unclear	Combines changed files, dependency relationships and relevant tests.
AI explanations may be unreliable	Grounds explanations in code evidence.
Static analysis may miss dynamic behavior	Labels uncertain findings instead of claiming certainty.

2.5 Competitive consideration

- GitHub Copilot already offers repository-aware assistance and pull-request review. This project must not be positioned as "AI explanations added to GitHub".
- Import-based impact analysis already exists in tools such as Nx `affected`, Jest `--findRelatedTests`, madge and dependency-cruiser.
- The intended combination is: visual exploration + PR impact analysis + evidence linked to real files and lines, inside GitHub.
- **No uniqueness claim** is made until the feature set is benchmarked against existing tools.

3. Scope and quality rules

In the MVP	Not in the MVP (future)
File-level import graph	Function-level calls, extends, uses
Changed files, affected files, related tests	Mapping diff lines to specific functions
Confirmed / inferred / unverified labels	Embedding-based candidate tests (labeled inferred)
Grounded LLM explanation of findings	Write or edit actions (only with explicit user approval)
JavaScript / TypeScript only	More languages

Quality rules that apply to every step

- Do not make the product a generic repository chatbot.
- Do not let the LLM invent relationships; ground every explanation in analysis results.
- Do not present inferred or uncertain findings as confirmed facts.
- Do not edit or write to the repository in the first version. Never execute repository code; only read it.
- Get a working file-level graph before attempting function-level analysis.

4. Architecture

The system has two parts. The extension is thin: it only detects the page and shows the interface. The backend does the heavy work.

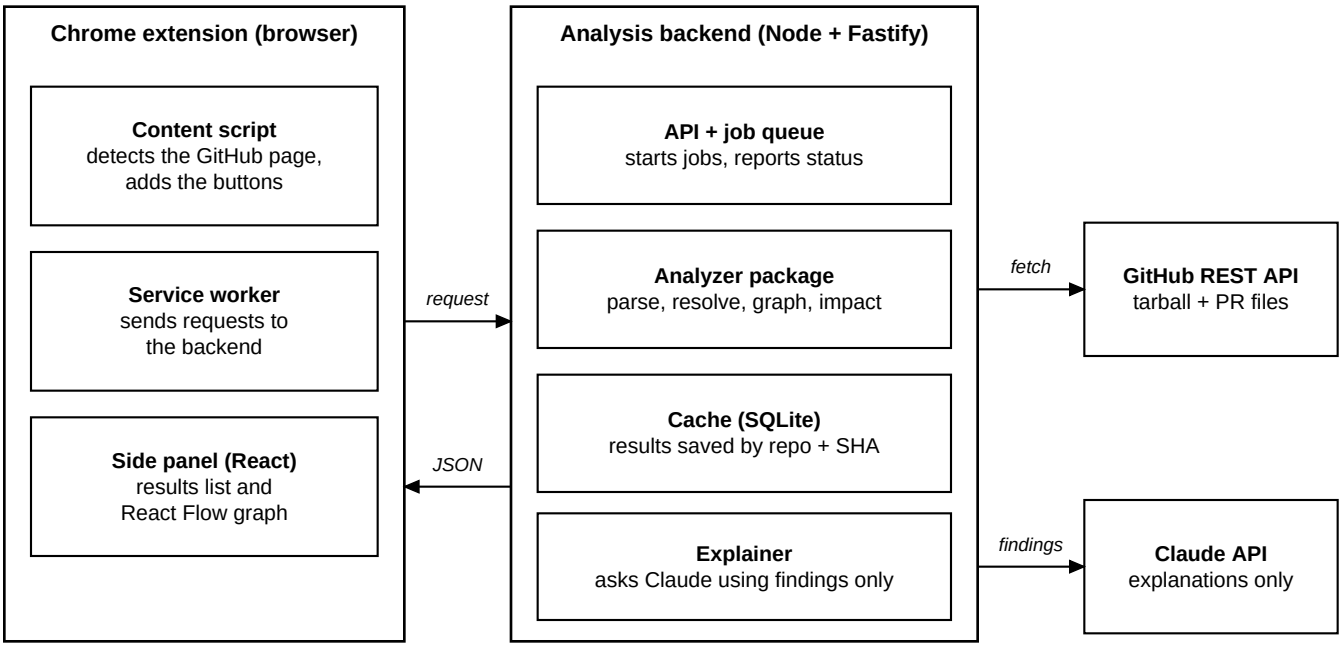


Figure 1. The two-part architecture. The extension talks only to the backend; the backend talks to GitHub and Claude.

Why split it?

- A browser extension cannot clone repositories, and parsing a large repository in the page would freeze the browser.
- API keys (GitHub token, Claude key) must never sit inside an extension. They stay on the backend.
- The analyzer is a separate package, so it can be tested from the command line without a browser.

5. Tech stack and why each piece is there

Part	Tool	Why (simple version)
Language	TypeScript	One language everywhere; types catch bugs early.
Extension	Chrome Manifest V3	The current required extension format.
Extension build	Vite + CRXJS	Fast builds and hot reload for extensions.
Extension UI	React + Tailwind	Easy side panel and result lists.
Graph UI	React Flow (@xyflow/react)	Ready-made interactive node-and-edge graphs.
Backend	Node.js + Fastify	Fast, simple HTTP API (Express also works).
Fetching code	GitHub REST API (tarball, pull request files)	Downloads an exact version of a repo without running git.
Parsing	Tree-sitter (JS/TS grammars)	Turns code into a tree, so imports are found reliably instead of by fragile text matching.
Import resolution	TypeScript compiler API (resolveModuleName)	Already understands tsconfig paths, aliases and index files, so you do not rewrite those rules.
Graph logic	graphology	JavaScript equivalent of NetworkX: nodes, edges, traversal.

Part	Tool	Why (simple version)
Cache	SQLite first (Postgres or Redis later)	Stores results by (repo, commit SHA); the same commit is never analyzed twice.
Background jobs	Simple in-process queue first	Big repos take time; the API returns a job id and the extension polls.
AI explanations	Claude API (backend only)	Turns structured findings into plain English.
Testing	Vitest + small fixture repos	Proves the analyzer is correct against known answers.
Packaging / CI	Docker, GitHub Actions	Same setup everywhere; tests run on every push.
Hosting	Railway or Render	Cheap or free public backend for the demo.

Deferred (not in the MVP): embeddings and Qdrant. If added later, they get one narrow job, such as suggesting tests that no import links to, always labeled *inferred*.

6. How the process unfolds: the 12-step pipeline

Steps 1 to 7 deliver repository exploration (Milestone 1). Steps 8 to 12 deliver pull-request impact (Milestone 2). Steps 3 to 6 run again for the pull request's head commit, and results are cached by commit SHA.

Solid box = repository flow (steps 1-7). Dashed box = pull-request flow (steps 8-12).

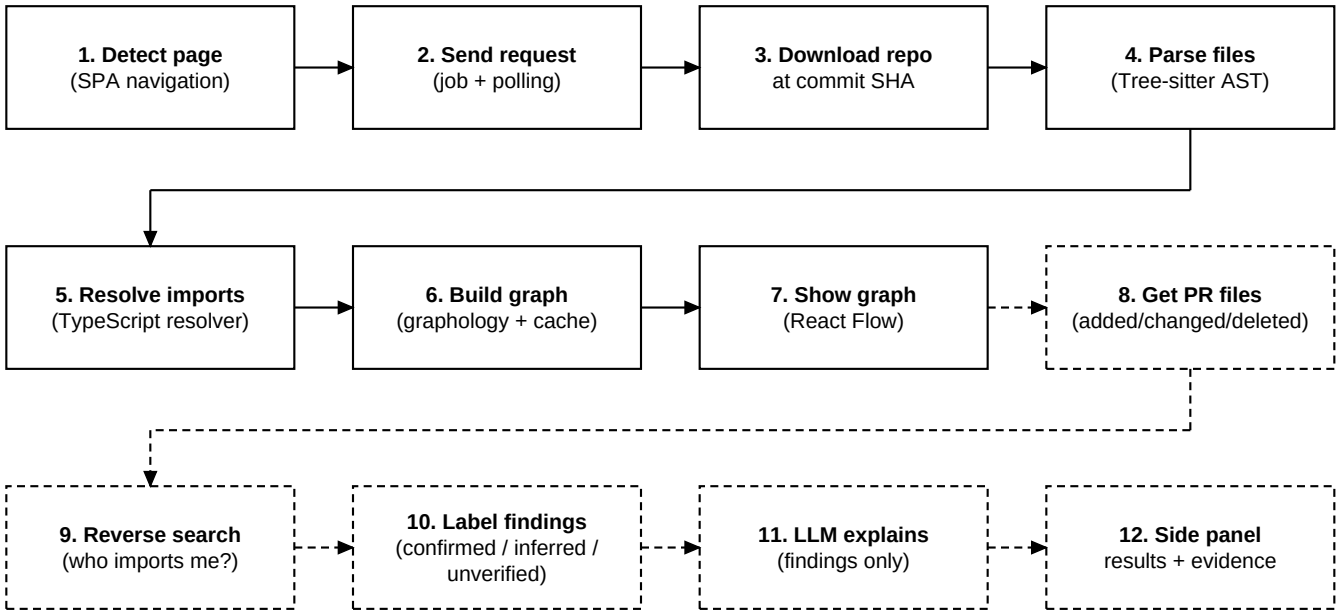


Figure 2. Overview of the pipeline from page detection to results.

Step 1 - Detect where the user is

What happens. GitHub does not reload the page when you click around. The extension listens for URL changes and decides which buttons to show.

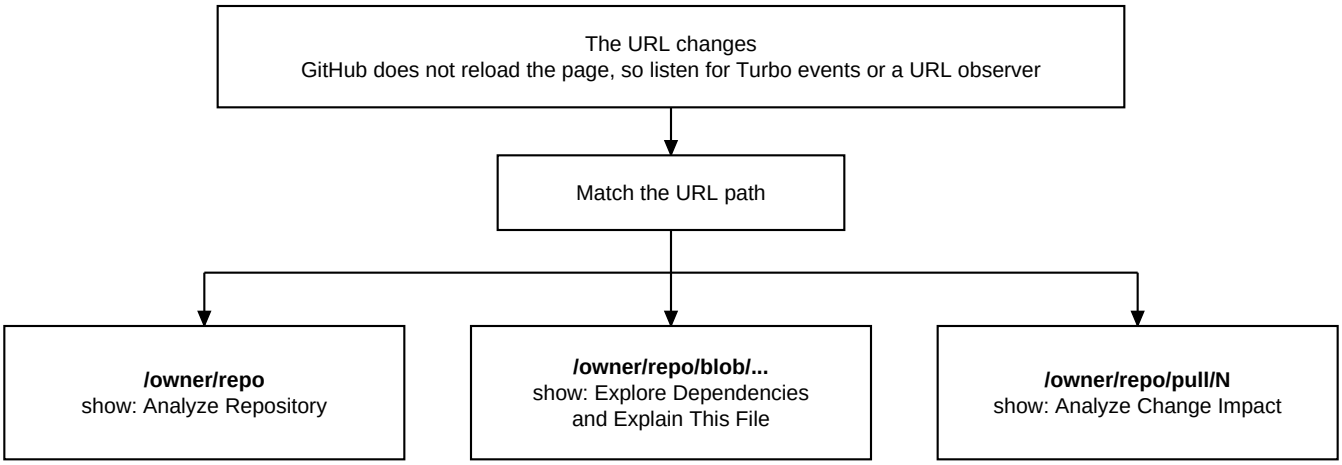


Figure 3. Page detection: the URL decides which action buttons appear.

Tools. Manifest V3, content script, TypeScript. **Why.** If detection ran only on page load, buttons would be missing or stale after navigation. **Concept:** single-page-app navigation.

Step 2 - Send a request and track it as a job

What happens. The extension sends the owner, repository and commit SHA (or pull request number) to the backend. The backend answers immediately with a job id, and the extension asks "done yet?" every one to two seconds. If the same repository and SHA are already in the cache, the result comes back at once.

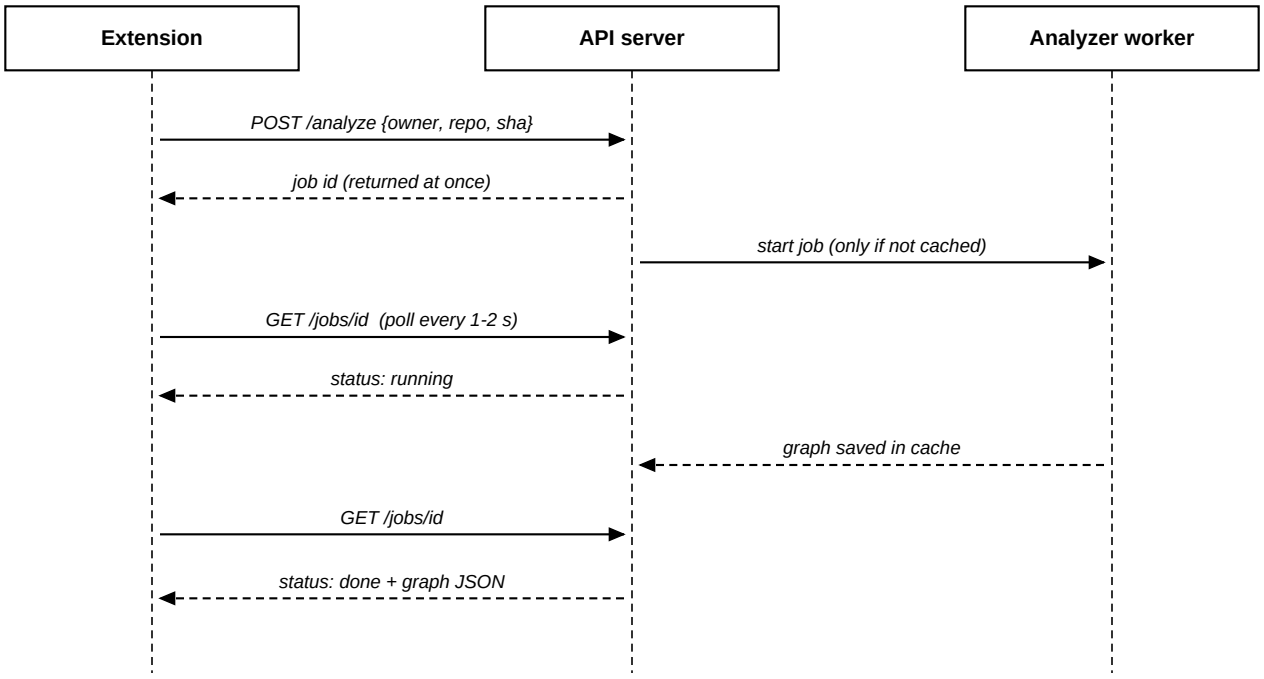


Figure 4. Asynchronous job with polling.

Tools. Fastify, an in-process job queue, SQLite cache. **Why.** Large repositories take seconds or minutes; holding one HTTP request open would time out and freeze the interface. **Concept:** asynchronous jobs, polling, caching.

Step 3 - Download the exact version and filter files

What happens. The backend downloads the repository as a tarball at a specific commit SHA (for a pull request, its head commit). It then filters the files:

Keep	Skip
.js .jsx .ts .tsx .mjs .cjs	node_modules, build output (dist, build), minified files, lockfiles, binaries, very large files

Limits on repository size and file count are enforced. **No code from the repository is ever executed.**

Tools. GitHub REST tarball endpoint. **Why.** A SHA gives exact, cacheable results; one tarball replaces hundreds of API calls; read-only handling is safe. **Concepts:** API integration, input validation, security limits.

Step 4 - Parse each file into a tree (AST)

What happens. Tree-sitter reads each file and turns it into a syntax tree. The code then finds every static import, every re-export (`export * from`), every `require()` call, and every dynamic `import()`. For each one it saves the path text, the line number and the kind.

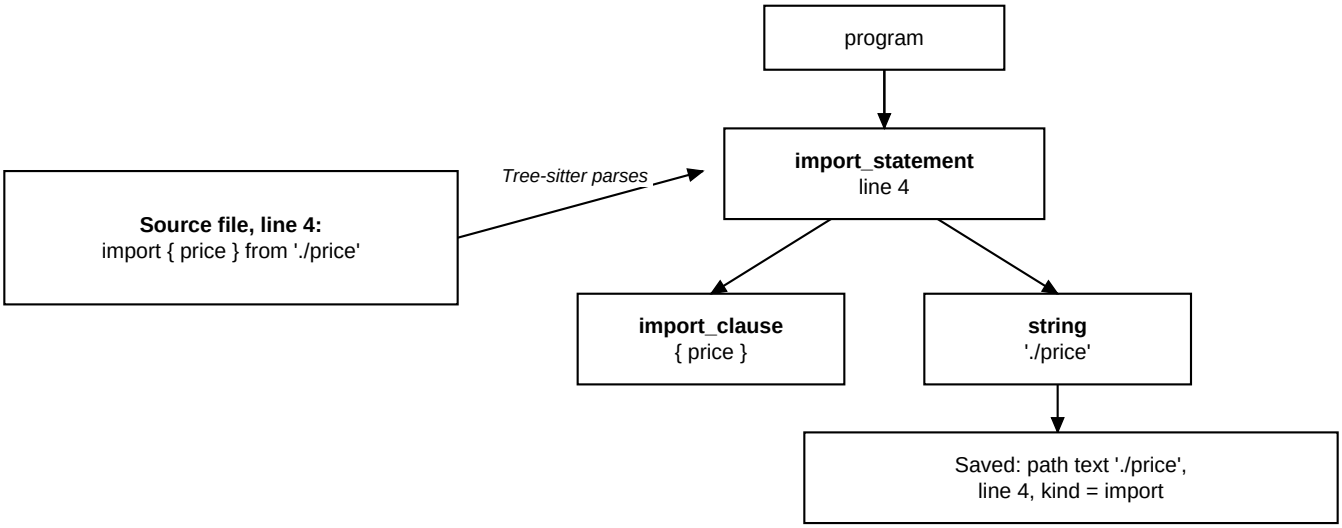


Figure 5. From a source line to a saved import record.

Why. Plain text search is fooled by comments and strings; a tree is exact. **Concept:** Abstract Syntax Trees, the same idea compilers and linters use.

Step 5 - Resolve imports to real files

What happens. A path like `'./price'` is only text. The backend must work out which file it means: `price.ts`? `price/index.ts`? Does `@/utils` point to `src/utils`? The TypeScript resolver answers this using the repository's `tsconfig.json`.

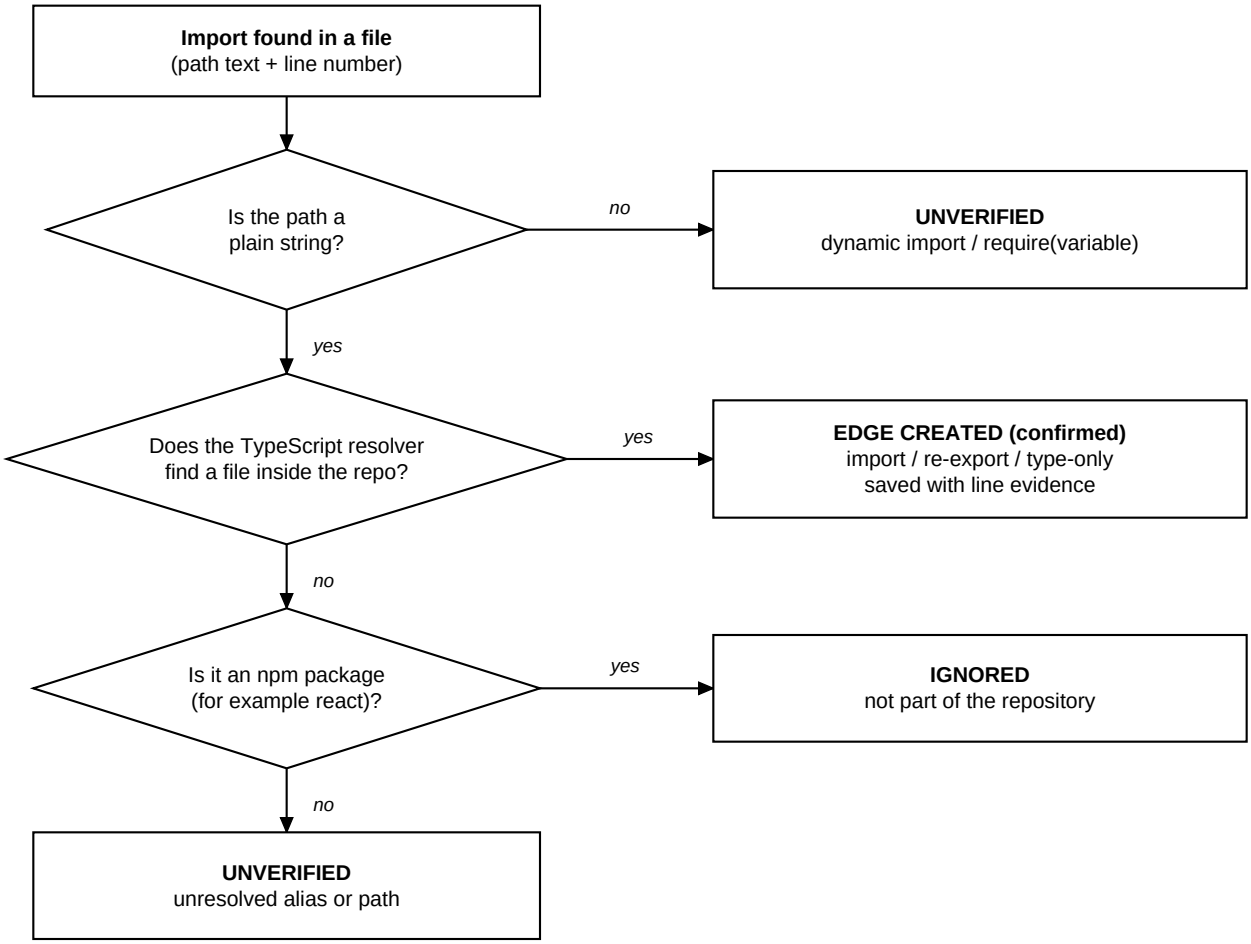


Figure 6. Decision flow for every import found.

Why this is the hardest part. Aliases (`paths`, `baseUrl`), barrel files (index files that re-export), `.js` to `.ts` mapping and monorepo workspaces all cause misses. Budget real time, test on fixture repositories, and record every failure as *unverified*. **Concept:** module resolution.

Step 6 - Build the dependency graph

What happens. Each file becomes a node. Each import becomes a directed edge: `checkout.ts -> price.ts` means "checkout imports price". Every edge stores its evidence, and the finished graph is cached by (repository, SHA). Files that import each other (cycles) are handled with a visited set.

Edge field	Meaning
from, to	The importing file and the imported file.
importLine	Line number of the import in the importing file.
specifier	The exact path text, such as ./price.
kind	import, re-export or type-only.
resolutionMethod	How the path was resolved (relative, alias, index file...).

Tool. graphology. **Concepts:** graph data structures, data modeling, caching.

Step 7 - Show the graph (Analyze Repository)

What happens. The backend returns nodes and edges as JSON and React Flow draws them. Clicking a node opens the file on GitHub at the right lines (a link ending in `#L10-L20`) and lists what it imports and what imports it. For large repositories, nodes are grouped by folder and expanded on click rather than drawn all at once.

Tools. React Flow, React, Tailwind. **Concepts:** data visualization, performance thinking.

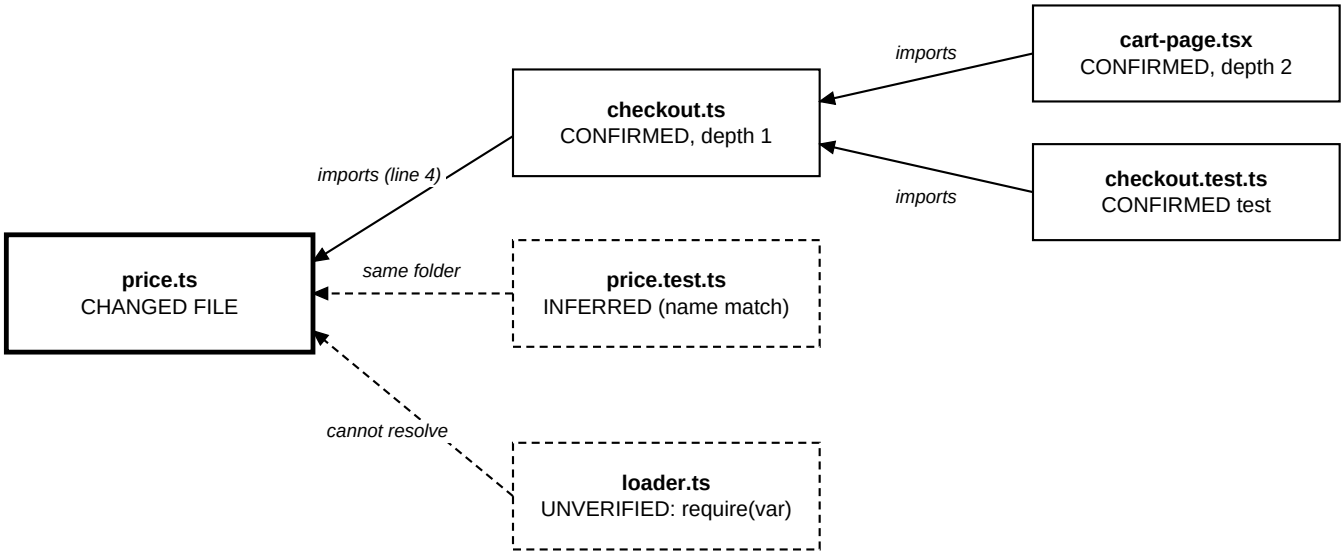
Step 8 - Find what changed in the pull request

What happens. The backend calls GitHub's "list pull request files" endpoint and handles all four file statuses:

Status	How it is handled
Added	Show its dependencies; nothing imported it before.
Modified	The main case: search for importers of the file.
Deleted	Everything that imported it may now be broken. The file is gone from the head commit, so its importers are looked up in the base commit's graph.
Renamed	Treated as a delete of the old path plus an add of the new path; importers of the old path need attention.

Step 9 - Compute the impact (reverse search)

What happens. A reverse breadth-first search starts at each changed file and asks "who imports me? who imports them?" up to a depth limit (for example 3). A visited set prevents infinite loops. Then tests are added: test files (`*.test.*`, `*.spec.*`, `__tests__`) found in the affected set, and test files that sit beside the changed file with a matching name.



Arrow = "imports". The impact search follows the arrows backwards from the changed file.
Solid arrow = resolved import. Dashed arrow = a guess or an unresolved link, never shown as fact.

Figure 7. Example: price.ts changes. The search walks the import arrows backwards.

Wording matters. A test that imports affected code is evidence the test exercises that code. It does not prove the test covers the changed lines. The UI says **"Tests that import affected code"**, not "tests that cover this change". **Concept:** graph algorithms (reverse BFS, depth limits, cycle handling).

Step 10 - Label every finding

Label	Rule	Example
Confirmed	Reached by following resolved import, require or re-export edges.	checkout.ts imports price.ts on line 4.
Inferred	A heuristic, with no direct import edge.	price.test.ts sits next to price.ts.
Unverified	A possible link the analyzer could not resolve.	require(someVariable) in loader.ts.

Each finding carries its **evidence chain**: the path of files and the file and line of every import on that path. The side panel shows the count of unverified links at the top ("3 links the tool could not resolve"), so users see the limits before trusting the results.

Step 11 - Let the LLM explain, under strict rules

What happens. Claude receives only the pull request diff, the list of findings and their evidence chains. The prompt says: explain why these may matter, use only the findings provided, do not mention unlisted files, keep the labels. The backend then checks the answer.

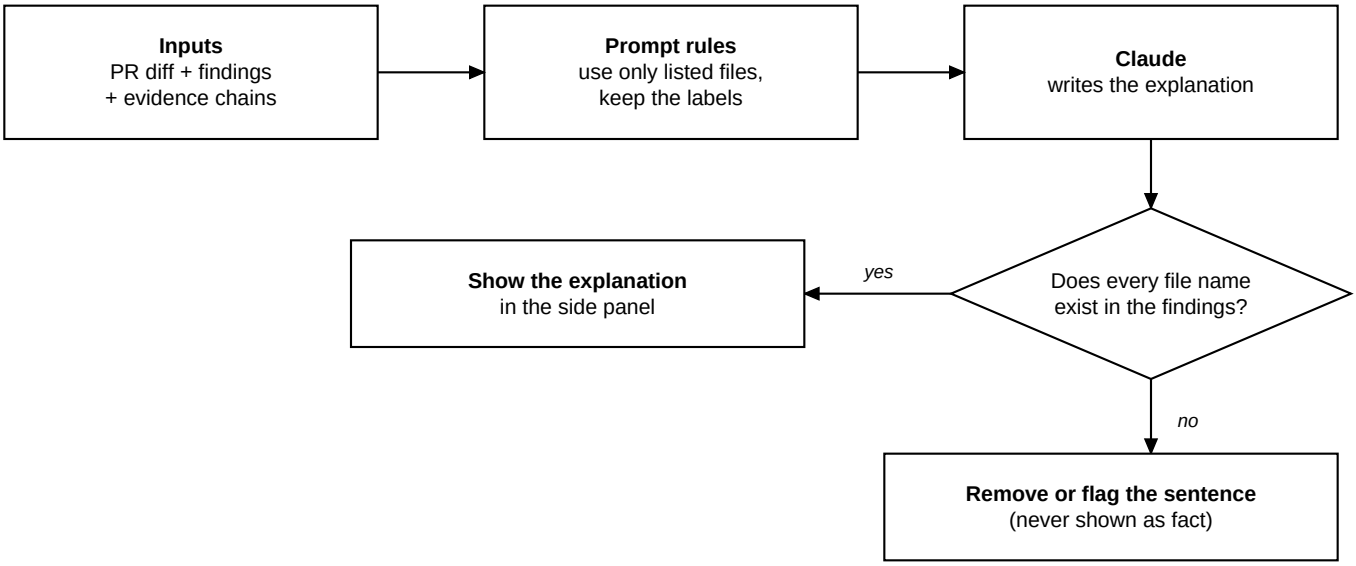


Figure 8. Grounded generation with an output check.

Concepts: grounded generation, guardrails, output validation. The API key stays on the backend.

Step 12 - Show results in a side panel

What happens. Results appear in a side panel or overlay while the user stays on the GitHub page. Findings are grouped by label. Each has its evidence chain and a "go to line" link. The extension only suggests and explains; it never changes the repository.

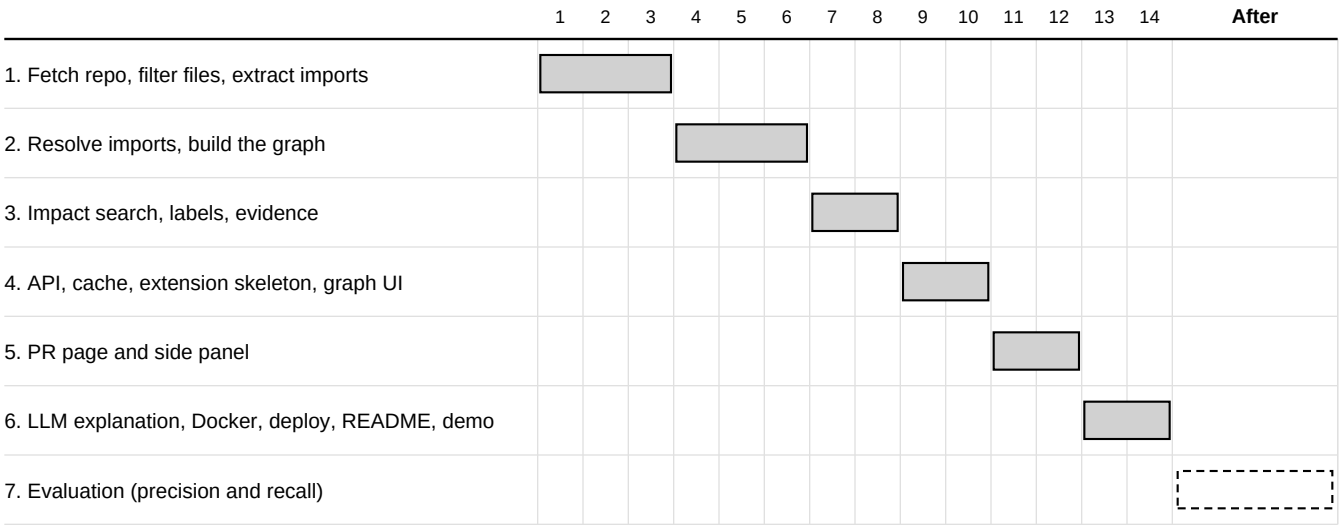
7. Project folder layout

```
/packages
  /analyzer     fetch, parse, resolve, graph, impact
                (pure logic, no web code, testable from the command line)
/apps
  /api          Fastify server, job handling, cache, LLM calls
  /extension    Manifest V3, content script, service worker, React UI
/fixtures       tiny sample repos with known correct answers
/eval           scripts that score accuracy on real pull requests
```

Why a separate analyzer? It can be tested without a browser, and it is the part of the project with the most engineering value.

8. Build order

Build the engine first, from the command line. Extension debugging is fiddly, and analysis logic and extension code should not be debugged at the same time.



Numbers = project days. Dashed bar = continues after the two-week build.

Figure 9. Two-week schedule, followed by the evaluation.

Days	Goal	You know it works when...
1-3	Fetch repo by SHA, filter files, extract imports with Tree-sitter	A command prints every import with its line number for a real repository.
4-6	Resolve imports (aliases, index files), build the graph	Results on 2-3 repositories match what you check by hand.
7-8	Reverse search, labels, evidence chains, test matching	Given a file, you get the right affected list and tests.
9-10	API, cache, job polling, extension skeleton, React Flow graph	Clicking Analyze Repository on GitHub draws a graph.
11-12	PR page: changed files, impact, side panel	Analyze Change Impact shows labeled findings.
13-14	LLM explanation with validation, Docker, deploy, README, demo	A public link and a recorded demo exist.
After	Evaluation	You have precision and recall numbers.

Milestone 1 - Repository exploration. Open a repository, click Analyze Repository, see a graph built from real files and dependencies, click a node to inspect the file or lines.

Milestone 2 - Pull-request impact. Open a pull request, click Analyze Change Impact, see changed files, affected modules, relevant tests and evidence, with inferred and unverified findings clearly marked.

9. Evaluation: what makes it resume-worthy

Without numbers, this is a demo. With numbers, it is engineering. The evaluation compares the tool against a naive baseline on real historical pull requests.

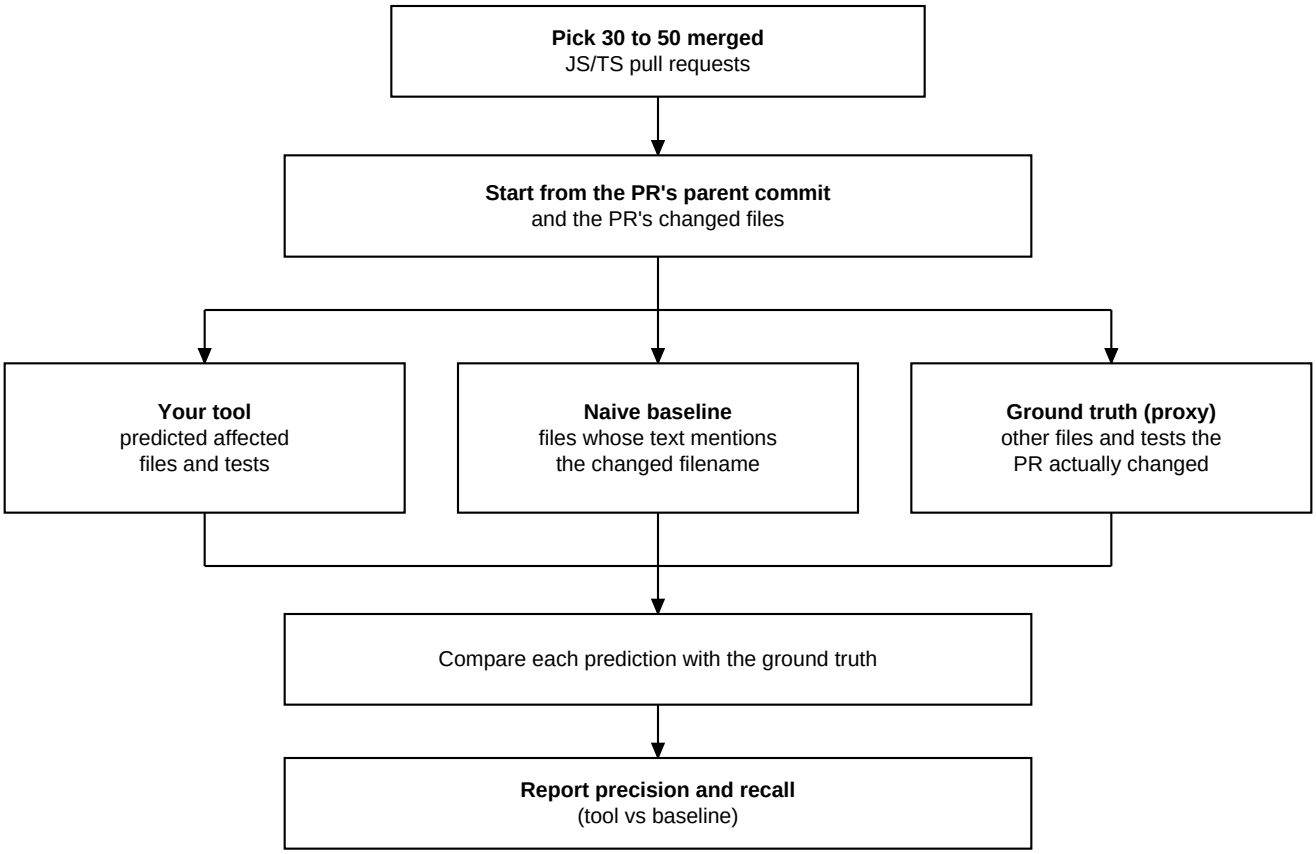


Figure 10. Evaluation procedure.

Precision = how many of the tool's suggestions were right. **Recall** = how many of the real ones it found.

Honest limit. A pull request's other changed files are a *proxy* for true impact, not perfect truth. State this in the README. Report only numbers you have actually measured.

10. Real-world concepts demonstrated

Concept	Where it appears
Static analysis and ASTs	Tree-sitter import extraction (step 4)
Module resolution	tsconfig paths, aliases, barrel files (step 5)
Graph algorithms	Reverse BFS, depth limits, cycle handling (steps 6, 9)
Client-backend design	Thin extension, analysis backend (section 4)
Async jobs and polling	Job id, status checks (step 2)
Caching	Results keyed by (repo, commit SHA) (steps 2, 6)
GitHub API	Tarball, PR files, added / deleted / renamed handling (steps 3, 8)

Concept	Where it appears
Browser extension architecture	Manifest V3, content script, service worker, SPA navigation (steps 1, 12)
Data visualization	React Flow graph (step 7)
Grounded LLM use	Findings-only prompt plus output validation (step 11)
Security	No code execution, size limits, secrets on the server (step 3)
Testing, CI, Docker, deployment	Vitest fixtures, GitHub Actions, containers (sections 5, 8)
Empirical evaluation	Precision and recall against a baseline (section 9)

11. Risks and how to handle them

Risk	Plan
Import resolution gets messy	Use the TypeScript resolver, test on fixtures, mark failures as unverified.
Huge repositories are slow	Size and file limits, a job queue, cache by SHA.
Graph is unreadable at scale	Folder grouping and expand-on-click.
GitHub rate limits	Backend token, caching, tarball instead of per-file calls.
Dynamic behavior is invisible	Say so in the UI and use the unverified label.
LLM makes things up	Give it only the findings and validate its output.

12. Success criteria

Problem	How you will know it is addressed
1. Understanding	A new user can find an entry point and a module's dependents within a minute.
2. Dependencies	Import edges match a manual check on fixture repositories (100% on fixtures).
3. PR impact	Precision and recall against historical PRs, versus the naive baseline.
4. Tests	Share of PRs where the tool surfaces the tests that were actually changed in the PR.
5. Trust	Every finding has an evidence chain, and no finding references a file that is not in the analysis.

Validation. Ask 3 to 5 developers to try the extension on a real pull request and record what they say. This gives honest feedback without claiming market proof.

13. Resume line and README notes

Resume bullet (fill in numbers only after measuring):

> Built a Chrome extension and TypeScript analysis backend that maps JS/TS repositories into dependency graphs and traces pull-request change impact with evidence-linked confirmed / inferred / unverified findings. Evaluated on [N] historical PRs: [X]% precision and [Y]% recall vs. a [Z]% keyword baseline.

README notes

- Write the "Problem" section as commonly reported developer difficulties, not statistics.
- Compare GitHub Copilot, Nx affected, madge and dependency-cruiser against the three differentiators (in-GitHub experience, evidence with labels, measured accuracy), and state plainly that import-based impact analysis already exists.